EXPERIMENT 1A:

```
#include <stdio.h>
#include <time.h>

int main () {
        int arr[100], i, n, c, found = 0;
        clock_t start, end;
        double cpu_time_used;

        printf("Enter N: ");
        scanf("%d", &n);

        printf("Enter Array of %d elements:\n", n);
        for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
        }

        printf("Enter Search element c:");
        scanf("%d", &c);

        start = clock();

        for (i = 0; i < n; i++) {
        if (arr[i] == c) {
        printf("Element %d found in position %d\n", c, i + 1);
        found = 1;
        break;
        }
        }

        end = clock();

        if (!found) {
        printf("Element %d not found\n", c);
        }

        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        printf("Execution time: %.6f Seconds\n", cpu_time_used);

   return 0; // Note: This function is non-standard (usually conio.h)
}
```

```
Enter N: 5
Enter Array of 5 elements:
2
4
6
8
10
Enter Search element c:4
Element 4 found in position 2
Execution time: 0.000015 Seconds
```

EXPERIMENT 1B:

```
#include <stdio.h>
#include <time.h>

int binarySearch(int arr[], int n, int key) {
        int low = 0;
        int high = n - 1;

        while (low <= high) {
        int mid = (high + low) / 2;

        if (arr[mid] == key) {
        return mid;
        } else if (arr[mid] < key) {
        low = mid + 1;
        } else {
        high = mid - 1;
        }
        }

        return -1;
}

int main() {
        int n = 100000;
        int arr[n];

        for (int i = 0; i < n; i++) {
        arr[i] = i;
        }

        int key = 99999;

        clock_t start, end;
        double cpu_time_used;

        start = clock();

        int result = binarySearch(arr, n, key);

        end = clock();

        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

        if (result != -1) {
        printf("Element found at index %d\n", result);
        } else {
        printf("Element not found\n");
        }
        printf("Execution time: %.6f seconds\n", cpu_time_used);
        return 0;     }
```

```
Element found at index 99999
Execution time: 0.000002 seconds
```

EXPERIMENT 1C:

```
#include <stdio.h>
#include <time.h>

int recursiveLinearSearch(int arr[], int size, int key, int index) {
        if (index == size) {
        return -1;
        }

        if (arr[index] == key) {
        return index;
        }

        return recursiveLinearSearch(arr, size, key, index + 1);
}

int main() {
        int n = 100000;
        int arr[n];

        for (int i = 0; i < n; i++) {
        arr[i] = i;
        }

        int key = 99999;

        clock_t start, end;
        double cpu_time_used;

        start = clock();

        // Start search from index 0
        int result = recursiveLinearSearch(arr, n, key, 0);

        end = clock();

        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

        if (result != -1) {
        printf("Element found at index %d\n", result);
        } else {
        printf("Element not found\n");
        }

        printf("Execution time: %.6f seconds\n", cpu_time_used);

        return 0;
}
```

```
Element found at index 99999
Execution time: 0.000002 seconds
```

EXPERIMENT 1D:

```
#include <stdio.h>
#include <time.h>

int recursiveBinarySearch(int arr[], int low, int high, int key) {
        if (low > high) {
        return -1; // Element not found
        }

        int mid = low + (high - low) / 2; // Safer mid calculation

        if (arr[mid] == key) {
        return mid;
        }
        else if (arr[mid] > key) {
        // Search the left half
        return recursiveBinarySearch(arr, low, mid - 1, key);
        }
        else {
        // Search the right half
        return recursiveBinarySearch(arr, mid + 1, high, key);
        }
}

int main() {
        int n = 100000;
        int arr[n];

        for (int i = 0; i < n; i++) {
        arr[i] = i;
        }

        int key = 99999;

        clock_t start, end;
        double cpu_time_used;

        start = clock();

        // Initial call: low=0, high=n-1
        int result = recursiveBinarySearch(arr, 0, n - 1, key);

        end = clock();
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
        if (result != -1) {
        printf("Element found at index %d\n", result);
        } else {
        printf("Element not found\n");
        }  return 0;    }
```

```
Element found at index 99999
Execution time: 0.000002 seconds
```

EXPERIMENT 2A:

```
#include <stdio.h>
#include <time.h>

void insertionSort(int array[], int n) {
	int i, element, j;

	for (i = 1; i < n; i++) {
	element = array[i];
	j = i - 1;

	// Move elements of array[0..i-1], that are greater than element,
	// to one position ahead of their current position
	while (j >= 0 && array[j] > element) {
	array[j + 1] = array[j];
	j = j - 1;
	}
	array[j + 1] = element;
	}
}

void printArray(int array[], int n) {
	int i;
	for (i = 0; i < n; i++) {
	printf("%d ", array[i]);
	}
	printf("\n");
}

int main() {
	int arr[] = {50, 23, 9, 18, 61, 32};

	clock_t start, end;
	double cpu_time_used;

	// Calculate the number of elements in the array
	int n = sizeof(arr) / sizeof(arr[0]);

	printf("\nBefore sorting: ");
	printArray(arr, n);

	start = clock();

	insertionSort(arr, n);
	end = clock();
	printf("\nAfter sorting: ");
	printArray(arr, n);
	cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
	printf("Execution time: %.6f seconds\n", cpu_time_used);
	return 0;
}
```

```
Before sorting: 50 23 9 18 61 32
After sorting: 9 18 23 32 50 61
Execution time: 0.000002 seconds
```

EXPERIMENT 2B:

```
#include <stdio.h>
#include <time.h>

// Define a structure to hold both maximum and minimum elements
struct pair {
	int max;
	int min;
};
struct pair maxMinDivideConquer(int arr[], int low, int high) {
	struct pair result, left, right;
	int mid;

	// Case 1: Small problem size (only 1 element)
	if (low == high) {
	result.max = arr[low];
	result.min = arr[low];
	return result;
	}

	// Case 2: Small problem size (only 2 elements)
	if (high == low + 1) {
	if (arr[low] < arr[high]) {
	result.min = arr[low];
	result.max = arr[high];
	} else {
	result.min = arr[high];
	result.max = arr[low];
	}
	return result;
	}

	// Case 3: Problem size > 2 (Divide and Conquer)
	mid = low + (high - low) / 2; // Calculate middle index

	// Divide: Recursively find max and min in the two halves
	left = maxMinDivideConquer(arr, low, mid);
	right = maxMinDivideConquer(arr, mid + 1, high);

	// Conquer: Combine the results

	// Find the overall maximum
	if (left.max > right.max) {
	result.max = left.max;
	} else {
	result.max = right.max;
	}

	// Find the overall minimum
	if (left.min < right.min) {
	result.min = left.min;
```

```
        } else {
        result.min = right.min;
        }

        return result;
}

int main() {
        // Array definition and size calculation
        int arr[] = {6, 4, 26, 14, 33, 64, 46};
        int n = sizeof(arr) / sizeof(arr[0]);

        // Timing variables
        clock_t start, end;
        double cpu_time_used;

        // Start timing
        start = clock();

        // Call the function
        // Pass the full array range: 0 to n-1
        struct pair result = maxMinDivideConquer(arr, 0, n - 1);

        // Stop timing
        end = clock();

        // Print results
    printf("Maximum element: %d\n", result.max);

    printf("Minimum element: %d\n", result.min);

        // Calculate time
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

        // Print execution time
    printf("Execution time: %.6f seconds\n", cpu_time_used);

        return 0;
}
```

```
Maximum element: 64
Minimum element: 4
Execution time: 0.000002 seconds
```

EXPERIMENT 3A:

```
#include <stdio.h>
#include <stdlib.h> // Required for malloc and free
#include <time.h>
void merge(int arr[], int left, int mid, int right) {
	int i, j, k;

	// Calculate the size of the two subarrays
	int n1 = mid - left + 1;
	int n2 = right - mid; // The number of elements in the right subarray

	// Create temporary arrays L and R
	// Using malloc to dynamically allocate memory
	int *L = (int *)malloc(n1 * sizeof(int));
	int *R = (int *)malloc(n2 * sizeof(int));

	// Copy data to temp arrays L[] and R[]
	for (i = 0; i < n1; i++) {
	L[i] = arr[left + i];
	}
	for (j = 0; j < n2; j++) {
	R[j] = arr[mid + 1 + j];
	}

	// Merge the temp arrays back into arr[left..right]
	i = 0; // Initial index of first subarray
	j = 0; // Initial index of second subarray
	k = left; // Initial index of merged subarray

	while (i < n1 && j < n2) {
	if (L[i] <= R[j]) {
	arr[k] = L[i];
	i++;
	} else {
	arr[k] = R[j];
	j++;
	}
	k++;
	}

	// Copy the remaining elements of L[], if any
	while (i < n1) {
	arr[k] = L[i];
	i++;
	k++;
	}

	// Copy the remaining elements of R[], if any
	while (j < n2) {
	arr[k] = R[j];
	j++;
```

```
        k++;
        }

        // Free the dynamically allocated memory
        free(L);
        free(R);
}
void mergeSort(int arr[], int left, int right) {
        if (left < right) {
        // Find the middle point
        int mid = left + (right - left) / 2;

        // Recursively sort the first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
        }
}
void printArray(int arr[], int size) {
        int i;
        for (i = 0; i < size; i++) {
        printf("%d ", arr[i]);
        }
        printf("\n");
}


int main() {
        int arr[] = {12, 11, 13, 5, 6, 7};

        clock_t start, end;
        double cpu_time_used;

        // Calculate the number of elements in the array
        int n = sizeof(arr) / sizeof(arr[0]);

        printf("Array is:\n");
        printArray(arr, n);

        start = clock();

        // Call mergeSort with the initial range (0 to n-1)
        mergeSort(arr, 0, n - 1);

        end = clock();

        printf("\nSorted:\n");
        printArray(arr, n);
```

```
        // Calculate time
        cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

        // Print execution time
        printf("\nExecution time: %.6f seconds\n", cpu_time_used);


    return 0;
}
```

```
Array is:
12 11 13 5 6 7

Sorted:
5 6 7 11 12 13

Execution time: 0.000005 seconds
```

EXPERIMENT 3B:

```
#include <stdio.h>
#include <time.h>
void swap(int* a, int* b) {
	int temp = *a;
	*a = *b;
	*b = temp;
}

int partition(int arr[], int low, int high) {
	int pivot = arr[high]; // Pivot is the last element
	int i = (low - 1);		// Index of smaller element

	for (int j = low; j <= high - 1; j++) {
	// If current element is smaller than or equal to pivot
	if (arr[j] <= pivot) {
	i++; // Increment index of smaller element
	swap(&arr[i], &arr[j]);
	}
	}
	// Swap the pivot element with the element at the index i+1
	swap(&arr[i + 1], &arr[high]);
	return (i + 1);
}
void quickSort(int arr[], int low, int high) {
	if (low < high) {
	// pIndex is partitioning index, arr[pIndex] is now at correct place
	int pIndex = partition(arr, low, high);

	// Separately sort elements before and after partition
	quickSort(arr, low, pIndex - 1);
	quickSort(arr, pIndex + 1, high);
	}
}
void printArray(int arr[], int size) {
	int i;
	for (i = 0; i < size; i++) {
	printf("%d ", arr[i]);
	}
	printf("\n");
}

int main() {
	int arr[] = {10, 7, 8, 9, 1, 5};

	clock_t start, end;
	double cpu_time_used;

	// Calculate the number of elements in the array
	int n = sizeof(arr) / sizeof(arr[0]);
```

```
    printf("Array:\n");
    printArray(arr, n);

    start = clock();

    // Call quickSort with the initial range (0 to n-1)
    quickSort(arr, 0, n - 1);

    end = clock();

    printf("\nSorted:\n");
    printArray(arr, n);

    // Calculate time
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Print execution time
    printf("Execution time: %.6f seconds\n", cpu_time_used);

    return 0;
}
```

```
Array:
10 7 8 9 1 5

Sorted:
1 5 7 8 9 10
Execution time: 0.000002 seconds
```

EXPERIMENT 4A:

```
#include <stdio.h>
#include <stdbool.h>
#include <time.h>

#define N 8  // Change this for different board sizes

static void printSolution(int board[N][N])
{
	for (int i = 0; i < N; ++i)
	{
	for (int j = 0; j < N; ++j)
	printf("%c ", board[i][j] ? 'Q' : '.');
	printf("\n");
	}
	printf("\n");
}

static bool isSafe(int board[N][N], int row, int col)
{
	// Check current row on the left side
	for (int i = 0; i < col; ++i)
	if (board[row][i])
	return false;

	// Check upper-left diagonal
	for (int i = row, j = col; i >= 0 && j >= 0; --i, --j)
	if (board[i][j])
	return false;

	// Check lower-left diagonal
	for (int i = row, j = col; i < N && j >= 0; ++i, --j)
	if (board[i][j])
	return false;

	return true;
}

static bool solveNQUtil(int board[N][N], int col)
{
	if (col >= N)
	return true; // All queens placed

	for (int i = 0; i < N; ++i)
	{
	if (isSafe(board, i, col))
	{
	board[i][col] = 1;
	if (solveNQUtil(board, col + 1))
	return true;
	board[i][col] = 0; // Backtrack
	}
	}
	return false;
}
```

```
static void solveNQ(void)
{
	int board[N][N];
	// Initialize to 0
	for (int i = 0; i < N; ++i)
	for (int j = 0; j < N; ++j)
	board[i][j] = 0;

	if (!solveNQUtil(board, 0))
	{
	printf("Solution does not exist.\n");
	return;
	}

	printSolution(board);
}

int main(void)
{
	clock_t start,end;
	double cpu_time_used;
	start=clock();
	solveNQ();
	end=clock();
	cpu_time_used = ((double)(end-start)) / CLOCKS_PER_SEC;
printf("Execution time: %.6f seconds\n", cpu_time_used);

	return 0;
}
```

```
Q . . . . . . .
. . . . . . Q .
. . . . Q . . .
. . . . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .

Execution time: 0.000102 seconds
```

EXPERIMENT 4B:

```
#include <stdio.h>
#include<time.h>
void findSubset(int set[], int subset[], int n, int index, int target, int currentSum) {  if (currentSum == target)
{
 printf("Subset found: ");
 for (int i = 0; i < index; i++) {
 printf("%d ", subset[i]);
 }
 printf("\n");
 return;
 }
 if (currentSum > target || n == 0) {
 return;
 }
 // Include the current element
 subset[index] = set[0];
 findSubset(set + 1, subset, n - 1, index + 1, target, currentSum + set[0]);
 // Exclude the current element
 findSubset(set + 1, subset, n - 1, index, target, currentSum); }
int main() {
 int n, target;
 clock_t start,end;
 double cpu_time_used;
 printf("Enter the number of elements in the set: ");  scanf("%d", &n);
 int set[n], subset[n];
 printf("Enter the elements of the set: ");  for (int i = 0; i < n; i++) {
 scanf("%d", &set[i]);
 }
 printf("Enter the target sum: ");
 scanf("%d", &target);
 printf("Subsets with sum %d:\n", target);
 start=clock();
 findSubset(set, subset, n, 0, target, 0);
 end=clock();
 cpu_time_used=((double)(end-start))/CLOCKS_PER_SEC;
 printf("Execution time:%f second",cpu_time_used);
 return 0;
}
```

```
Enter the number of elements in the set: 4
Enter the elements of the set: 2
3
5
6
Enter the target sum: 5
Subsets with sum 5:
Subset found: 2 3
Subset found: 5
Execution time:0.000032 second
```

EXPERIMENT 5A:

```
#include <stdio.h>
#include <time.h>

// Function to find the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

// Function to solve the 0/1 Knapsack problem
int knapsack(int W, int wt[], int val[], int n) {
    int i, w;
    // Create a 2D array to store results of subproblems
    // knap[i][w] will store the maximum value for 'i' items and capacity 'w'
    int knap[n + 1][W + 1];

    // Build the knap table in a bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            // Base case: If no items or no capacity, value is 0
            if (i == 0 || w == 0) {
                knap[i][w] = 0;
            } else if (wt[i - 1] <= w) {
                // Take the maximum of two cases:
                // 1. Include the current item: val[i-1] + knap[i-1][w - wt[i-1]]
                // 2. Exclude the current item: knap[i-1][w]
                knap[i][w] = max(val[i - 1] + knap[i - 1][w - wt[i - 1]], knap[i - 1][w]);
            } else {
                // If current item's weight is more than current capacity, exclude it
                knap[i][w] = knap[i - 1][w];
            }
        }
    }
    // The maximum value will be in knap[n][W]
    return knap[n][W];
}

// Driver code to test the knapsack function
int main() {
    clock_t start, end;
    double cpu_time_used;
    int val[] = {60, 100, 120}; // Values of items
    int wt[] = {10, 20, 30};    // Weights of items
    int W = 50;                 // Maximum capacity of the knapsack
    int n = sizeof(val) / sizeof(val[0]); // Number of items

    start = clock();
    printf("The maximum value that can be put in the knapsack is: %d\n", knapsack(W, wt, val,
n));
```

```
    end = clock();

    cpu_time_used = ((double)(end - start) / CLOCKS_PER_SEC);
    printf("Execution time: %f seconds\n", cpu_time_used);

    return 0;
}
```

```
The maximum value that can be put in the knapsack is: 220
Execution time: 0.000048 seconds
```

EXPERIMENT 5B:

```
#include <stdio.h>
#include <stdlib.h>
#include<time.h>
// Structure to represent an item
typedef struct {
 int itemId;
 int weight;
 int profit;
 float pByw; // Profit-to-weight ratio
} Item;
// Comparison function for sorting items by profit-to-weight ratio in descending order
int compareItems(const void *a, const void *b) {
 Item *itemA = (Item *)a;
 Item *itemB = (Item *)b;
 if (itemA->pByw < itemB->pByw) return 1;
 if (itemA->pByw > itemB->pByw) return -1;
 return 0;
}
// Function to solve the Fractional Knapsack problem
float fractionalKnapsack(Item *items, int n, int capacity) {
 // Sort items based on profit-to-weight ratio in descending order
 qsort(items, n, sizeof(Item), compareItems);
 float totalProfit = 0.0;
 int currentWeight = 0;
 for (int i = 0; i < n; i++) {
 // If the current item can be taken completely
 if (currentWeight + items[i].weight <= capacity) {
 currentWeight += items[i].weight;
 totalProfit += items[i].profit;
 printf("Added item %d (Weight: %d, Profit: %d) completely. Current weight: %d, Total  profit:
%.2f\n",
 items[i].itemId, items[i].weight, items[i].profit, currentWeight, totalProfit);  } else {
 // Take a fraction of the current item
 float remainingCapacity = capacity - currentWeight;
 float fraction = remainingCapacity / items[i].weight;
 totalProfit += fraction * items[i].profit;
 currentWeight += remainingCapacity; // Knapsack is now full
 printf("Added %.2f%% of item %d (Weight: %d, Profit: %d). Current weight: %d, Total  profit:
%.2f\n",
 fraction * 100, items[i].itemId, items[i].weight, items[i].profit, currentWeight,  totalProfit);
 break; // Knapsack is full, no more items can be added
 }
 }
 return totalProfit;
}
int main() {
    clock_t start, end;
    double cpu_time_used;
```

```
 int n, knapsackCapacity;
 printf("Enter the number of items: ");
 scanf("%d", &n);
 Item *items = (Item *)malloc(sizeof(Item) * n);
 if (items == NULL) {
 printf("Memory allocation failed.\n");
 return 1;
 }
 printf("Enter itemId, weight, and profit for each item:\n");  for (int i = 0; i < n; i++) {
 printf("Item %d: ", i + 1);
 scanf("%d %d %d", &items[i].itemId, &items[i].weight, &items[i].profit);  items[i].pByw =
(float)items[i].profit / items[i].weight;  }
 printf("Enter the knapsack capacity: ");
 scanf("%d", &knapsackCapacity);
 start=clock();
 float maxProfit = fractionalKnapsack(items, n, knapsackCapacity);  printf("\nMaximum profit for
the given capacity: %.2f\n", maxProfit);
 free(items);
     end = clock();

    cpu_time_used = ((double)(end - start) / CLOCKS_PER_SEC);
    printf("Execution time: %f seconds\n", cpu_time_used);

 return 0;   }
```

```
Enter the number of items: 2
Enter itemId, weight, and profit for each item:
Item 1: 2
3
4
Item 2: 5
6
7
Enter the knapsack capacity: 9
Added item 2 (Weight: 3, Profit: 4) completely. Current weight: 3, Total  profit: 4.00
Added item 5 (Weight: 6, Profit: 7) completely. Current weight: 9, Total  profit: 11.00

Maximum profit for the given capacity: 11.00
Execution time: 0.000047 seconds
```

EXPERIMENT 6A

```c
#include <stdio.h>
#include<time.h>
int main() {
clock_t start, end;
double cpu_time_used;
 int n, t1 = 0, t2 = 1, nextTerm;
 // Get the number of terms from the user
 printf("Enter the number of terms: ");
 scanf("%d", &n);
 // Print the first two terms (0 and 1)
 start=clock();
 printf("Fibonacci Series: %d, %d", t1, t2);
 // Generate and print subsequent terms
 for (int i = 3; i <= n; ++i) {
 nextTerm = t1 + t2; // Calculate the next term
 printf(", %d", nextTerm); // Print the next term
 t1 = t2; // Update t1 to the previous t2
 t2 = nextTerm; // Update t2 to the newly calculated
     nextTerm
 }
 printf("\n"); // Newline for cleaner output
 end = clock();
 cpu_time_used = ((double)(end - start) /
     CLOCKS_PER_SEC);
printf("Execution time: %f seconds\n", cpu_time_used);
return 0;
}
```

```
Enter the number of terms: 5
Fibonacci Series: 0, 1, 1, 2, 3
Execution time: 0.000018 seconds


=== Code Execution Successful ===
```

EXPERIMENT 6B

```c
#include <stdio.h>
#include <string.h>
#include<time.h>
// Function to find the maximum of two integers
int max(int a, int b) {
 return (a > b) ? a : b;
}
// Function to find the Longest Common Subsequence (LCS)
int lcs(char *X, char *Y, int m, int n) {
 int L[m + 1][n + 1]; // DP table to store lengths of LCS
 // Fill the L table in bottom-up manner
 for (int i = 0; i <= m; i++) {
 for (int j = 0; j <= n; j++) {
```

```
 if (i == 0 || j == 0) {
 L[i][j] = 0; // Base case: if either string is empty, LCS is 0
 } else if (X[i - 1] == Y[j - 1]) {
 L[i][j] = L[i - 1][j - 1] + 1; // Characters match, add 1 to diagonal
 } else {
 L[i][j] = max(L[i - 1][j], L[i][j - 1]); // Characters don't match, take max of above or left
 }
 }
 }
 // L[m][n] contains the length of LCS. Now, reconstruct the LCS string.
 int index = L[m][n];
 char lcs_str[index + 1];
 lcs_str[index] = '\0'; // Null-terminate the string
 int i = m, j = n;
 while (i > 0 && j > 0) {
 if (X[i - 1] == Y[j - 1]) {
 lcs_str[index - 1] = X[i - 1]; // Character is part of LCS
 i--;
 j--;
 index--;
 } else if (L[i - 1][j] > L[i][j - 1]) {
 i--; // Move up
 } else {
 j--; // Move left
 }
 }
 printf("Longest Common Subsequence: %s\n", lcs_str);
 return L[m][n]; // Return the length of LCS
}
int main() {
clock_t start, end;
double cpu_time_used;
 char X[] = "AGGTAB";
 char Y[] = "GXTXAYB";
 int m = strlen(X);
 int n = strlen(Y);
 start=clock();
 printf("Length of LCS is %d\n", lcs(X, Y, m, n));
 end = clock();
cpu_time_used = ((double)(end - start) / CLOCKS_PER_SEC);
printf("Execution time: %f seconds\n", cpu_time_used);
return 0;
}
```

```
Longest Common Subsequence: GTAB
Length of LCS is 4
Execution time: 0.000044 seconds
```

EXPERIMENT 7A:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // Include for time measurement

// Global variables
int i, j, k, a, b, u, v, n, ne = 1;
int min, mincost = 0;
int cost[9][9];
int parent[9] = {0}; // Initialized to zero

// Function declarations
int find(int);
int uni(int, int);

// Find the root of the set containing element i
int find(int i)
{
   while (parent[i])
      i = parent[i];
   return i;
}

// Union the sets containing elements i and j. Returns 1 if a union occurs, 0 otherwise.
int uni(int i, int j)
{
   if (i != j)
   {
      parent[j] = i;
      return 1;
   }
   return 0;
}

int main(void) // Standard C main function
{
   printf("Kruskal's algorithm in C\n");
   printf("========================\n");

   printf("Enter the no. of vertices (max 8):\n");
   if (scanf("%d", &n) != 1 || n < 1 || n >= 9) return 1;

   printf("\nEnter the cost adjacency matrix (0 for no edge):\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         if (scanf("%d", &cost[i][j]) != 1) return 1;
         // Use 999 for no edge
         if (cost[i][j] == 0)
```

```
            cost[i][j] = 999;
        }
    }

    // START TIMING
    clock_t start = clock();

    printf("The edges of Minimum Cost Spanning Tree are\n");

    // Kruskal's main loop: continues until n-1 edges are added
    while (ne < n)
    {
        // Step 1: Find the minimum cost edge (a, b)
        for (i = 1, min = 999; i <= n; i++)
        {
            for (j = 1; j <= n; j++)
            {
                if (cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }

        // Step 2: Check for cycle using DSU operations
        u = find(u);
        v = find(v);

        if (uni(u, v)) // If union is successful (no cycle)
        {
            printf("%d edge (%d,%d) = %d\n", ne++, a, b, min);
            mincost += min;
        }

        // Step 3: Mark the edge as processed
        cost[a][b] = cost[b][a] = 999; // Use 999 to mark as processed
    }

    // STOP TIMING
    clock_t end = clock();
    double cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

    printf("\nMinimum cost = %d\n", mincost);
    printf("Execution time: %.6f seconds\n", cpu_time_used);

    return 0;
}
```

```
Kruskal's algorithm in C
========================
Enter the no. of vertices (max 8):
2

Enter the cost adjacency matrix (0 for no edge):
4
6
8
10
The edges of Minimum Cost Spanning Tree are
1 edge (1,2) = 6

Minimum cost = 6
Execution time: 0.000031 seconds
```

EXPERIMENT 7B:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // Include for time measurement

#define INFINITY 9999
#define MAX 10

void dijkstra(int G[MAX][MAX], int n, int startnode);

int main(void)
{
   int G[MAX][MAX], i, j, n, u;

   printf("Enter no. of vertices (max %d): ", MAX);
   if (scanf("%d", &n) != 1 || n <= 0 || n > MAX)
   {
      fprintf(stderr, "Invalid number of vertices.\n");
      return 1;
   }

   printf("\nEnter the adjacency matrix (0 for no edge or self-loop):\n");
   for (i = 0; i < n; i++)
   {
      for (j = 0; j < n; j++)
      {
         if (scanf("%d", &G[i][j]) != 1)
         {
            fprintf(stderr, "Input error.\n");
            return 1;
         }
      }
   }

   printf("\nEnter the starting node (0 to %d): ", n - 1);
   if (scanf("%d", &u) != 1 || u < 0 || u >= n)
   {
      fprintf(stderr, "Invalid starting node.\n");
      return 1;
   }

   dijkstra(G, n, u);

   return 0;
}

void dijkstra(int G[MAX][MAX], int n, int startnode)
{
   // Arrays for algorithm state
```

```
int cost[MAX][MAX], distance[MAX], pred[MAX];
int visited[MAX], count, mindistance, nextnode, i, j;

// START TIMING
clock_t start_time = clock();

// Create the cost matrix: replace 0 (no edge) with INFINITY
for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      if(G[i][j] == 0)
         cost[i][j] = INFINITY;
      else
         cost[i][j] = G[i][j];
   }
}

// Initialize pred[], distance[] and visited[]
for(i = 0; i < n; i++)
{
   // Initial distance from startnode to i is the direct edge cost
   distance[i] = cost[startnode][i];
   pred[i] = startnode;
   visited[i] = 0;
}

distance[startnode] = 0;
visited[startnode] = 1;
count = 1;

// The loop runs n-1 times to find shortest paths to n-1 nodes
while(count < n) // Corrected termination condition: needs to process n nodes (count < n)
{
   mindistance = INFINITY;

   // Step 1: Find the next unvisited node with the minimum distance
   for(i = 0; i < n; i++)
   {
      if(distance[i] < mindistance && !visited[i])
      {
         mindistance = distance[i];
         nextnode = i;
      }
   }

   // Check if all remaining nodes are unreachable (distance remains INFINITY)
   if (mindistance == INFINITY)
      break;

   // Step 2: Mark the found node as visited
```

```
        visited[nextnode] = 1;

        // Step 3: Update distances to its neighbors through the nextnode
        for(i = 0; i < n; i++)
        {
            if(!visited[i])
            {
                if(mindistance + cost[nextnode][i] < distance[i])
                {
                    distance[i] = mindistance + cost[nextnode][i];
                    pred[i] = nextnode;
                }
            }
        }
        count++;
    }

    // STOP TIMING
    clock_t end_time = clock();
    double cpu_time_used = ((double) (end_time - start_time)) / CLOCKS_PER_SEC;

    // Print the path and distance of each node
    printf("\nShortest paths from node %d:\n", startnode);
    for(i = 0; i < n; i++)
    {
        if(i != startnode)
        {
            printf("\nDistance to node %d = %d", i, distance[i]);
            printf("\nPath: %d", i);

            // Print path by backtracking using the predecessor array
            j = i;
            if (distance[i] != INFINITY)
            {
                do
                {
                    j = pred[j];
                    printf(" <- %d", j);
                } while(j != startnode);
            }
        }
    }

    printf("\n\nExecution time: %.6f seconds\n", cpu_time_used);
}
```

```
Enter no. of vertices (max 10): 2

Enter the adjacency matrix (0 for no edge or self-loop):
4
6
8
10

Enter the starting node (0 to 1): 1

Shortest paths from node 1:

Distance to node 0 = 8
Path: 0 <- 1

Execution time: 0.000002 seconds
```

EXPERIMENT 8:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // For clock() and CLOCKS_PER_SEC

// Structure to represent a point
typedef struct
{
    int first;
    int second;
} Pair;

// Global variable to store the center of the polygon
Pair mid;

// Function to determine the quadrant of a point relative to 'mid'
int quad(Pair p)
{
    // The point p here represents the difference (p - mid)
    if (p.first >= 0 && p.second >= 0)
        return 1;
    if (p.first <= 0 && p.second >= 0)
        return 2;
    if (p.first <= 0 && p.second <= 0)
        return 3;
    return 4; // p.first >= 0 && p.second <= 0
}

// Function to check the orientation of three points (cross-product sign)
// Returns: 1 (CCW), 0 (Collinear), -1 (CW)
int orientation(Pair a, Pair b, Pair c)
{
    long long res = (long long)(b.second - a.second) * (c.first - b.first) - (long long)(c.second -
b.second) * (b.first - a.first);
    if (res == 0)
        return 0;
    if (res > 0)
        return 1; // Counter-Clockwise (Left Turn)
    return -1;    // Clockwise (Right Turn)
}

// Compare function for sorting by Polar Angle around 'mid'
int compare(const void *p1, const void *q1)
{
    Pair *p = (Pair *)p1;
    Pair *q = (Pair *)q1;
    Pair p_diff = {p->first - mid.first, p->second - mid.second};
    Pair q_diff = {q->first - mid.first, q->second - mid.second};
```

```
    int one = quad(p_diff);
    int two = quad(q_diff);

    // Sort by quadrant first
    if (one != two)
        return (one < two) ? -1 : 1;

    // Within the same quadrant, use cross product to determine angle
    // Cross product: (p - mid) x (q - mid)
    // Positive result means p is CCW from q (smaller angle), so p comes first (-1)
    long long cross_product = (long long)p_diff.second * q_diff.first - (long long)q_diff.second *
p_diff.first;

    if (cross_product == 0) return 0; // Collinear

    return (cross_product > 0) ? -1 : 1; // CCW (-1) or CW (1)
}

// Compare function for initial sorting by X-coordinate (and Y as tie-breaker)
int compare_x(const void *p1, const void *q1)
{
    Pair *p = (Pair *)p1;
    Pair *q = (Pair *)q1;

    if (p->first != q->first)
        return p->first - q->first;
    return p->second - q->second;
}

// Function to merge two convex hulls
Pair *merger(Pair *a, int n1, Pair *b, int n2, int *ret_size) {
    // ... (merger function body is unchanged) ...
    // Find rightmost point on A (ia) and leftmost point on B (ib)
    int ia = 0, ib = 0;
    for (int i = 1; i < n1; i++)
        if (a[i].first > a[ia].first)
            ia = i;
    for (int i = 1; i < n2; i++)
        if (b[i].first < b[ib].first)
            ib = i;

    // Find Upper Tangent
    int inda = ia, indb = ib;
    int done = 0;
    while (!done)
    {
        done = 1;
        // Move inda CCW on A until b[indb] -> a[inda] -> a[next] is CW or Collinear (>= 0)
        while (orientation(b[indb], a[inda], a[(inda + 1) % n1]) >= 0)
            inda = (inda + 1) % n1;
```

```
    // Move indb CW on B until a[inda] -> b[indb] -> b[prev] is CW or Collinear (<= 0)
    while (orientation(a[inda], b[indb], b[(n2 + indb - 1) % n2]) <= 0)
    {
        indb = (n2 + indb - 1) % n2;
        done = 0;
    }
}
int uppera = inda, upperb = indb;

// Find Lower Tangent
inda = ia;
indb = ib;
done = 0;
while (!done)
{
    done = 1;
    // Move indb CCW on B until a[inda] -> b[indb] -> b[next] is CW or Collinear (>= 0)
    while (orientation(a[inda], b[indb], b[(indb + 1) % n2]) >= 0)
        indb = (indb + 1) % n2;

    // Move inda CW on A until b[indb] -> a[inda] -> a[prev] is CW or Collinear (<= 0)
    while (orientation(b[indb], a[inda], a[(n1 + inda - 1) % n1]) <= 0)
    {
        inda = (n1 + inda - 1) % n1;
        done = 0;
    }
}
int lowera = inda, lowerb = indb;

// Construct the merged hull
Pair *ret = (Pair *)malloc((n1 + n2) * sizeof(Pair));
int k = 0;

// Trace from uppera to lowera on A
int ind = uppera;
ret[k++] = a[uppera];
while (ind != lowera)
{
    ind = (ind + 1) % n1;
    ret[k++] = a[ind];
}

// Trace from lowerb to upperb on B
ind = lowerb;
ret[k++] = b[lowerb];
while (ind != upperb)
{
    ind = (ind + 1) % n2;
    ret[k++] = b[ind];
}
```

```
    *ret_size = k;

    // Free the memory of the sub-hulls
    free(a);
    free(b);

    // Reallocate to the correct size
    Pair *final_ret = (Pair *)realloc(ret, k * sizeof(Pair));
    // Check for realloc failure (though unlikely here)
    if (final_ret == NULL) {
        // Handle error if realloc fails
        *ret_size = 0;
        return NULL;
    }
    return final_ret;
}

// Brute force algorithm to find the convex hull for a small set of points
Pair *bruteHull(Pair *a, int n, int *ret_size)
{
    // ... (bruteHull function body is unchanged, but corrected for dynamic allocation) ...
    // Note: The logic for calculating mid (centroid) in this function is unusual (multiplying
coordinates by k before division)
    // but retained to match the original intent. The main logic for edge checking is standard
O(N^3) or O(N^2).

    Pair *temp_hull = (Pair *)malloc(n * sizeof(Pair));
    int k = 0;

    // Brute force: Check every segment
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            // Line equation: a1*x + b1*y + c1 = 0
            long long x1 = a[i].first, x2 = a[j].first;
            long long y1 = a[i].second, y2 = a[j].second;
            long long a1 = y1 - y2;
            long long b1 = x2 - x1;
            long long c1 = x1 * y2 - y1 * x2;

            int pos = 0, neg = 0;
            // Check which side of the line all other points lie
            for (int m = 0; m < n; m++)
            {
                long long val = a1 * a[m].first + b1 * a[m].second + c1;
                if (val <= 0)
                    neg++;
                if (val >= 0)
                    pos++;
            }
```

```
        // If all points are on one side (pos == n OR neg == n), the segment is a hull edge
        if (pos == n || neg == n)
        {
            int already_added;

            // Add point i if not already in the hull
            already_added = 0;
            for (int l = 0; l < k; l++)
            {
                if (temp_hull[l].first == a[i].first && temp_hull[l].second == a[i].second)
                    already_added = 1;
            }
            if (!already_added)
                temp_hull[k++] = a[i];

            // Add point j if not already in the hull
            already_added = 0;
            for (int l = 0; l < k; l++)
            {
                if (temp_hull[l].first == a[j].first && temp_hull[l].second == a[j].second)
                    already_added = 1;
            }
            if (!already_added)
                temp_hull[k++] = a[j];
        }
    }
}

*ret_size = k;

// Prepare for polar angle sort (calculate centroid 'mid')
Pair *ret = (Pair *)realloc(temp_hull, k * sizeof(Pair)); // Final hull array
if (ret == NULL && k > 0) return NULL; // Handle realloc failure
if (k == 0) { *ret_size = 0; return NULL; } // Handle empty hull

mid.first = 0;
mid.second = 0;
for (int i = 0; i < k; i++)
{
    mid.first += ret[i].first;
    mid.second += ret[i].second;
    // The following lines in the original code are unnecessary and potentially problematic:
    // ret[i].first *= k;
    // ret[i].second *= k;
}

// Sort the hull points by polar angle around the centroid
qsort(ret, k, sizeof(Pair), compare);
```

```
    // The following loop from the original code is unnecessary after removing the 'multiply by k'
lines
    /*
    for (int i = 0; i < k; i++)
    {
        ret[i].first /= k;
        ret[i].second /= k;
    }
    */

    return ret;
}

// Function to divide the set of points and recursively find the convex hull
Pair *divide(Pair *a, int n, int *ret_size)
{
    // The input array 'a' is a segment of the original points,
    // it must be copied for the recursive calls to manage memory correctly.

    // Copy the current segment of points for the left half
    Pair *left_copy = (Pair *)malloc((n / 2) * sizeof(Pair));
    for(int i = 0; i < n / 2; i++) {
        left_copy[i] = a[i];
    }

    // Copy the current segment of points for the right half
    Pair *right_copy = (Pair *)malloc((n - n / 2) * sizeof(Pair));
    for(int i = 0; i < n - n / 2; i++) {
        right_copy[i] = a[n / 2 + i];
    }

    if (n <= 5)
    {
        // For the base case, use the original segment's address and size,
        // as bruteHull handles its own memory allocation/reallocation.
        Pair *hull = bruteHull(a, n, ret_size);
        free(left_copy); // Free the copies that won't be used
        free(right_copy);
        return hull;
    }

    int mid_idx = n / 2;
    Pair *left_hull;
    Pair *right_hull;
    int left_size, right_size;

    // Recursively find hulls of the copied segments
    left_hull = divide(left_copy, mid_idx, &left_size);
    right_hull = divide(right_copy, n - mid_idx, &right_size);

    // Free the copies after they have been used to generate the sub-hulls
```

```
    free(left_copy);
    free(right_copy);

    // Merge the two hulls (merger frees the sub-hulls)
    return merger(left_hull, left_size, right_hull, right_size, ret_size);
}

// Driver code
int main()
{
    Pair a[] = {{0, 0}, {1, -4}, {-1, -5}, {-5, -3}, {-3, -1},
                {-1, -3}, {-2, -2}, {-1, -1}, {-2, -1}, {-1, 1}};
    int n = sizeof(a) / sizeof(a[0]);

    clock_t start, end;
    double cpu_time_used;

    // 1. Initial Sort: The 'divide' step requires points to be sorted by X-coordinate
    qsort(a, n, sizeof(Pair), compare_x);

    // 2. Start Timing
    start = clock();

    // 3. Find the Convex Hull
    int ret_size;

    // Create a copy of 'a' since 'divide' and 'merger' will manage/free memory
    Pair *a_copy = (Pair *)malloc(n * sizeof(Pair));
    for(int i = 0; i < n; i++) a_copy[i] = a[i];

    Pair *ans = divide(a_copy, n, &ret_size);

    // 4. End Timing
    end = clock();

    // 5. Output Result and Time
    printf("Convex hull vertices:\n");
    if (ans != NULL) {
        for (int i = 0; i < ret_size; i++)
        {
            printf("(%d, %d)\n", ans[i].first, ans[i].second);
        }
        free(ans);
    } else {
        printf("Could not compute convex hull.\n");
    }

    free(a_copy); // Free the copy

    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nExecution time: %f seconds\n", cpu_time_used);
```

```
    return 0;
}
```

```
Convex hull vertices:
(-3, -1)
(-2, -1)
(-1, -5)
(1, -4)
(-1, 1)

Execution time: 0.000020 seconds
```

EXPERIMENT 9:

```
#include <stdio.h>
#include <stdlib.h>
#include<time.h>
// Function to perform modular exponentiation
long long power_mod(long long base, long long exp, long long mod) {  long long result = 1;
 base = base % mod;
 while (exp > 0) {
 if (exp % 2 == 1) {
 result = (result * base) % mod;
 }
 exp = exp >> 1;
 base = (base * base) % mod;
 }
 return result;
}
// Miller-Rabin test for a single iteration
int miller_rabin_test(long long n, long long d) {
 long long a = 2 + rand() % (n - 4); // Random base in range [2, n-2]
 long long x = power_mod(a, d, n);
 if (x == 1 || x == n - 1) {
 return 1; // Probably prime
 }
 while (d != n - 1) {
 x = (x * x) % n;
 d *= 2;
 if (x == 1) return 0; // Composite
 if (x == n - 1) return 1; // Probably prime
 }
 return 0; // Composite
}
// Miller-Rabin Primality Test
int is_prime(long long n, int k) {
 if (n <= 1 || n == 4) return 0; // Not prime
 if (n <= 3) return 1; // Prime
 long long d = n - 1;
 while (d % 2 == 0) {
 d /= 2;
 }
 for (int i = 0; i < k; i++) {
 if (!miller_rabin_test(n, d)) {
 return 0; // Composite
 }
 }
 return 1; // Probably prime
}
int main() {
 long long n;
 clock_t start,end;
```

```
 double cpu_time_used;
 int k = 5; // Number of iterations for accuracy
 start=clock();
 printf("Enter a number to check if it is prime: ");  scanf("%lld", &n);
 if (is_prime(n, k)) {
 printf("%lld is a prime number.\n", n);  } else {
 printf("%lld is not a prime number.\n", n);  }
 end=clock();
 cpu_time_used-((double)(end-start))/CLOCKS_PER_SEC;
 printf("Execution time:%f second",cpu_time_used);
 return 0;
}
```

```
Enter a number to check if it is prime: 4
4 is not a prime number.
Execution time:0.000000 second
```